



MCU-Friendly Mamba
Optimizing Mamba Model Inference on Microcontrollers

Bartosz Drabiński¹

Supervisor: Bo Yang¹
Responsible Professor: Qing Wang¹

¹EEMCS, Delft University of Technology, The Netherlands

A Thesis Submitted to EEMCS Faculty Delft University of Technology,
In Partial Fulfilment of the Requirements
For the Bachelor of Computer Science and Engineering
June 2, 2026

Name of the student: Bartosz Drabiński
Final project course: CSE3000 Research Project
Thesis committee: dr. Qing Wang, Mark Neerinx

An electronic version of this thesis is available at <http://repository.tudelft.nl/>.

Abstract

The usage of machine learning is becoming increasingly more widespread in various domains. While state-of-the-art models continue to grow in size and computational cost, Edge AI targets devices with strict limits on processing power and memory. TinyML — the branch of machine learning focused on running models on microcontrollers — exemplifies these constraints. Meeting these constraints requires new model designs and careful adaptation of existing architectures. The Mamba architecture, built around State-Space Models, is a promising candidate due to its compact parameterization and strong performance on long-context tasks. In this paper we survey recent efforts to deploy Mamba on microcontrollers. We discuss possible Mamba deployment frameworks. Using TensorFlow Lite Micro we evaluate architecture modifications and optimization strategies tailored to MCU constraints. Our experiments show that while quantization can reduce memory usage, it may increase latency on hardware without efficient INT8 support. We also propose a splitting the model into smaller parts to avoid loop unrolling. Our findings show that Mamba is a promising architecture for TinyML tasks on microcontrollers, but that more work is needed to optimize the State-Space Model implementation for these devices.

Keywords: TinyML, Mamba, microcontroller, Edge AI

1 Introduction

Machine learning has recently enabled solutions to problems that were difficult or infeasible with traditional algorithms. This progress produced both large, general-purpose models (e.g., large language models) and many compact, task-specific models.

Most deployments still rely on centralized data centres, which are efficient but impose limitations: they require network connectivity, add latency, and raise privacy concerns.

Edge AI reduces these limitations by running inference locally on user devices. TinyML focuses on the extreme end of this spectrum, designing models that run on microcontrollers (MCUs). MCUs are severely resource-constrained: typical devices provide one or two CPU cores (usually clocked below 240 MHz) and on the order of 512 KB of RAM, so they cannot host large models or their full-precision weights. Low latency is also essential because many TinyML applications (for example, audio or video streams) require real-time processing.

Numerous model families have been developed for different trade-offs, including multilayer perceptrons (MLPs), convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Transformers. The recently proposed Mamba architecture [1] has emerged as a promising alternative to Transformers for long-context tasks. Mamba combines ideas from CNNs and RNNs and leverages Selective

State-Space Model (S6) block to achieve a smaller memory footprint while maintaining strong performance on long sequences.

Prior work has explored optimizations for Mamba such as quantization [2], pruning, and knowledge distillation [3]. MambaLite-Micro [4] implemented PyTorch-style inference for Mamba directly in C and ran it on ESP32-S3 and STM32 microcontrollers; this preserves compatibility but makes architecture changes and further optimizations cumbersome.

This paper asks the following question: How can Mamba be adapted and optimized for efficient inference on microcontrollers? To answer it, we evaluate deployment strategies, identify memory and latency bottlenecks, and measure the impact of GPU-oriented optimizations when applied to MCUs.

Section 2 surveys related work and background. Section 3 describes the optimizations and architectural changes investigated by us. Section 4 describes the embedded deployment setup. Section 5 reports the experimental results and then discusses them. Section 6 discusses ethical considerations. Finally, Section 7 concludes and outlines future work.

2 Related Work and Background

This section reviews TinyML constraints and recent Mamba work relevant to MCU deployment. We summarize common TinyML optimizations (quantization, pruning, distillation), then focus on Mamba’s SSM-based design and prior deployment attempts that motivate our contributions.

2.1 TinyML

TinyML concerns the design, optimization, and deployment of machine learning models on highly resource-constrained embedded devices such as microcontrollers. Models for these devices are typically trained with the target constraints in mind and then optimized for inference, since typical devices often have less than 256 KB of Data RAM (DRAM).

These constraints limit which architectures are practical. Common choices include convolutional neural networks (CNNs) and recurrent neural networks (RNNs), which can be more parameter-efficient than multilayer perceptrons (MLPs) [5].

MCUs have limited computational power, so latency is a critical concern. Many TinyML applications (for example, audio or accelerometer streams) require real-time processing, which makes models that rely on expensive mathematical operations harder to deploy. This is especially true for activation functions using trigonometric terms (e.g., Tanh) or exponentiation (e.g., Softmax, SiLU) [6].

The limited or slow floating-point support on many MCUs makes quantization an important optimization strategy. Quantization reduces the precision of weights and activations from 32-bit floats to lower-bit formats (for example, 8-bit or 4-bit integers), which can substantially reduce model size and memory usage. These gains can come at the cost of accuracy, and memory savings are not always proportional due to format and graph overheads [7]. In some cases, careless quantization can lead to large accuracy drops, especially for models not designed with quantization in mind [2].

Other strategies include pruning, knowledge distillation, and low-rank factorization [8]. Pruning removes redundant parameters, while knowledge distillation transfers knowledge from a larger teacher model into a smaller student. These techniques can be effective but are often more complex to apply, and reported size reductions vary by method and task [3].

2.2 Mamba architecture

The Mamba architecture [1] is a recent proposal that blends convolutional and recurrent design ideas to capture long-range dependencies while keeping a smaller memory footprint than Transformers. A top-level diagram of a Mamba block appears in Figure 1.

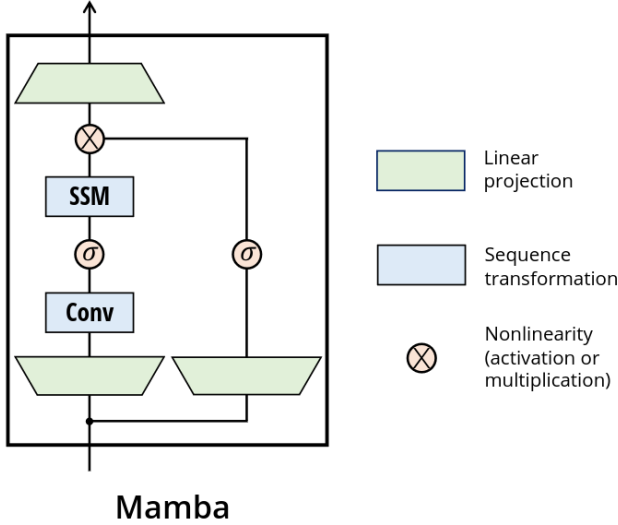


Figure 1: Diagram of the Mamba block

Mamba’s main building block is a Selective State-Space Model (S6) layer, which captures long-range structure while avoiding the quadratic complexity of attention. The S6 formulation can be written as

$$\begin{aligned} h_t &= \bar{A}h_{t-1} + \bar{B}x_t \\ y_t &= Ch_t + Dx_t \end{aligned}$$

In Mamba the S6 matrices are conditioned on the input, allowing the dynamics to adapt over time. S6 layers can be implemented using efficient scan (parallel-prefix) algorithms that parallelize well on modern hardware, but their recurrent-style structure can make them sensitive to quantization and numerical issues on deeply constrained hardware [2; 9].

Most prior work reducing Mamba model sizes targets mobile or embedded GPUs rather than MCUs. For example, Physics-Guided Tiny-Mamba Transformer [10] is reduced to 0.5M parameters for Jetson Nano, Tiny-ViM [11] targets 5M parameters for vision, and FEMBA [9] applies aggressive quantization at 7.8M parameters. These models remain far larger than typical MCU budgets.

Some efforts (e.g., LightMamba [12]) explore Selective State-Space Models for resource-constrained tasks such as

ultra-wideband positioning, but demonstrations on actual microcontrollers are limited. MambaLite-Micro [4] implements PyTorch-style inference for Mamba in C and runs on ESP32-S3 and STM32 MCUs, preserving compatibility but making architecture changes and further optimizations more cumbersome. Overall, deployment tooling for Mamba on MCUs remains immature.

2.3 Research gap

Two primary gaps motivate this work. First, many TinyML optimization techniques exist, but their applicability to Mamba is not well understood on MCU-class hardware. Second, while MambaLite-Micro demonstrates feasibility, it does not explore architecture modifications or deployment strategies that could further reduce memory usage and latency on MCUs. This paper evaluates deployment strategies, identifies Mamba-specific bottlenecks on microcontrollers, and proposes targeted improvements.

3 Proposed Mamba-MCU Optimizations

This section presents the deployment choices and architecture adaptations designed to fit Mamba on microcontrollers. We first justify the chosen toolchain and hardware, then describe the model reimplementations and the optimizations evaluated (quantization, looped SSM, activation simplifications).

3.1 Embedded Deployment

Deployment of machine learning models on microcontrollers, while feasible, still presents challenges. The MambaLite-Micro authors chose a path that preserves exact PyTorch inference results by extracting weights and hardcoding them in C. This approach preserves compatibility but makes architecture changes and further optimizations cumbersome, since any model modification requires C changes.

For this paper we decided not to take that route and instead selected an existing model-porting tool. We investigated frameworks such as TensorFlow Lite for Microcontrollers (TFLM, also referred to as LiteRT) [13] and ExecuTorch [14]. The hardware-agnostic workflow of TFLM provided more flexibility for experimentation, better documentation, and fewer deployment issues. By using TFLM we avoid additional intermediary formats (for example, we do not convert to ONNX), removing conversion differences as a factor in the experiments.

There are several tools built into and around TFLM. We use ‘tflm_model_transforms’ to strip redundant and debug data from the model, enabling fairer comparisons of model sizes by reducing graph overhead. We also use the AI Edge Quantizer, which provides precise control over quantization and operates directly on TFLite model files. The different deployment options are illustrated in Figure 2.

We also considered other deployment options such as Brevitas [15] (used by FEMBA) and TorchAO [16], which support quantization on PyTorch models and Quantization-Aware Training (QAT). Brevitas can target very low bit-widths. We did not use these tools in the final implementation because QAT often requires kernel fusion that is difficult to achieve efficiently on microcontrollers.

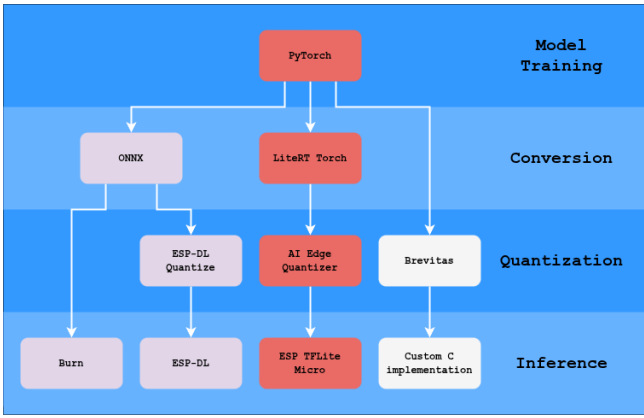


Figure 2: Diagram of the deployment steps and some options for a TinyML

Due to time constraints we implemented and tested models only on the ESP32 (Figure 3) family of microcontrollers (ESP32 and ESP32-S3) using ‘esp-tflite-micro’, a vendor-specific implementation of TFLM. We avoided vendor-specific pipelines such as ESP-DL or STM32Cube.AI so results would not be bound to a single MCU platform.

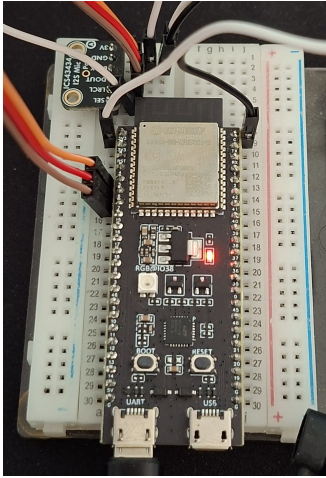


Figure 3: Photo of an ESP32S3 with ICS43434 microphone module attached

3.2 Model Architecture

We do not explore extensive hyperparameter search in this work; instead, we adopt the MambaLite-Micro architecture [4] so comparisons remain meaningful and our effort can focus on deployment and optimization. The architecture comprises a fully connected input layer, a single Mamba block, an average-pooling layer, and a fully connected output layer. The Mamba block input size is 64 with an expansion ratio of 2.

Because the original Mamba implementation relies on custom GPU kernels not available in TFLM, we re-implemented the architecture in PyTorch. This gives a more flexible deployment pipeline and makes it easier to modify activations

and reduce expensive exponentiation operations prior to conversion.

3.3 Deployment Optimization

For deployment optimization we focus primarily on quantization, evaluating INT8 weight and activation quantization.

Because TFLM lacks a general looping mechanism, the SSM implementation is unrolled by default, causing model size to grow linearly with sequence length. This is problematic for longer sequences (for example, the Speech Commands dataset). To mitigate this, we split the deployment model into three parts: pre-SSM, SSM step, and post-SSM. The inference code loops over the SSM step, reusing the same weights for each time step and significantly reducing model size. We compare both unrolled and looped implementations on the HAR dataset to study trade-offs.

4 Experimental Setup and Evaluation

This section describes datasets, preprocessing, evaluation metrics, and measurement procedures used to quantify accuracy, peak RAM, and latency. We provide enough detail to allow replication of conversion, quantization, and microcontroller benchmarking steps.

4.1 Dataset and Preprocessing

Many datasets have been used and demonstrated as viable for TinyML applications; these typically use sensor data such as audio, images, or accelerometer signals. For this work we use two datasets: the Human Activity Recognition (HAR) dataset [17] and the Google Speech Commands dataset [18].

These datasets are popular in the TinyML community and represent different data modalities (accelerometer and audio), which enables comparison with prior work such as MambaLite-Micro [4].

The HAR dataset consists of recordings of volunteers performing activities of daily living while carrying a waist-mounted smartphone with embedded inertial sensors. The data is preprocessed into 561 features. Following previous work, we pad this input to 570 features and split them into 10 time steps of 57 features each. State-of-the-art models can achieve nearly 98% accuracy on this dataset [19].

For the Google Speech Commands dataset we preprocess audio into Mel-frequency cepstral coefficients (MFCCs). We extract 40 MFCC features per time step, producing 51 time steps with 40 features each. While some implementations reduce the number of classes from 35 to 10, we kept all 35 classes to exercise long-sequence behavior. State-of-the-art models can achieve around 96% accuracy on this dataset [20].

4.2 Experimental Design

The experimental design evaluates parts of the deployment pipeline: model architecture, optimization strategies, and deployment choices. We measure both peak memory usage and latency, which are critical for TinyML.

Measuring peak memory and latency on an embedded device is challenging, but TFLM provides tools to help. For memory usage we use the TFLM memory recorder to measure peak memory and to separate permanently allocated versus temporary buffers. For latency we adapted the default

TFLM profiler to measure total inference time and time per operation type, which helps identify bottlenecks and the impact of optimizations.

For accuracy testing we run the full test set on the development PC for both the PyTorch implementation and after conversion to TFLite to ensure conversion does not introduce significant degradation. We also measure accuracy after quantization to quantify its impact.

On the microcontroller we evaluate accuracy on a small subset (50 samples) from each test set due to resource constraints, which provides a sanity check that the final embedded implementation matches expected behavior.

5 Results and Discussion

This section presents experimental results for accuracy, model sizes, latency, and memory usage, followed by interpretation and discussion of practical trade-offs when deploying Mamba on MCUs.

5.1 Model accuracy

We first trained models on a development PC using a custom PyTorch implementation of the Mamba block. Training produced validation accuracies of 98.6% for HAR and 90.09% for the KWS task.

We then converted the models to TFLite and evaluated accuracy before and after quantization. Results are shown in Table 1 for both the original (full) model and the split model.

Table 1: Accuracy results for full and split models.

Dataset	Float32	Int8
HAR PyTorch	93.89%	-
HAR TFLite Full	93.89%	93.59%
HAR TFLite Split	93.89%	92.37%
KWS PyTorch	88.96%	-
KWS TFLite Full	88.96%	85.06%
KWS TFLite Split	88.96%	84.10%

We also measured output file sizes after each optimization step (Table 2). Raw file size can be misleading for formats that include graph metadata in addition to raw weights: quantize/dequantize operators and preserved names can outweigh tensor-size savings. Using ‘tflm_model.transforms’ to strip debug and name metadata reduced file size by up to 30% in our experiments.

Table 2: Model sizes before and after quantization and optimization.

Dataset	Original	Quantized	Optimized
HAR Full	195 KB	102 KB	70.5 KB
HAR Split*	160 KB	62 KB	50.9 KB
KWS Full	319 KB	267 KB	—**
KWS Split*	165 KB	63 KB	51.9 KB

*Sizes of the 3 partial models have been added up

**Concatenation operation too long to handle optimization.

It is often assumed that storing model data in flash is much slower than storing it in RAM. We measured a simple array-sum benchmark with data in DRAM versus flash on the ESP32 and found flash access 2.15x slower for this task (far smaller than typical desktop disparities). Note that TFLM copies relevant arrays from ROM to RAM on tensor initialization, so this initial-copy overhead is separate from steady-state inference behavior.

5.2 Embedded Performance

The next important metrics are the inference time (latency) and RAM usage on the microcontroller.

Although quantized models are often faster due to reduced floating-point work, for our TFLM Mamba models the INT8 variants are slower (Figure 4). INT8 latency is over 2x larger than float in many cases. This is primarily because the ESP32 lacks hardware acceleration for vector int8 operations, so quantized multiplications are implemented via multiple 32-bit operations (adds, multiplies, bit shifts, comparisons). The model split increases latency slightly but not substantially.

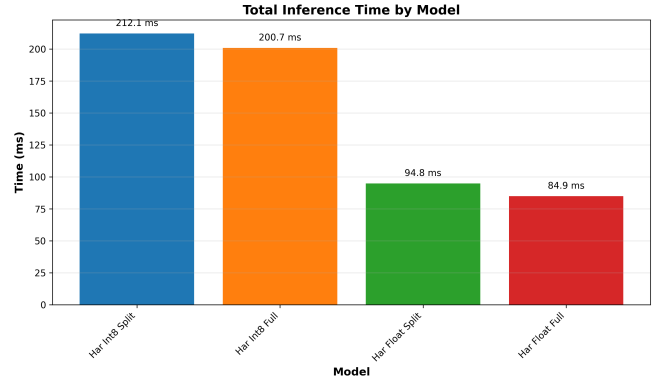


Figure 4: Graph of the latency of the HAR models

We measured per-operation latency breakdowns (Figure 5).

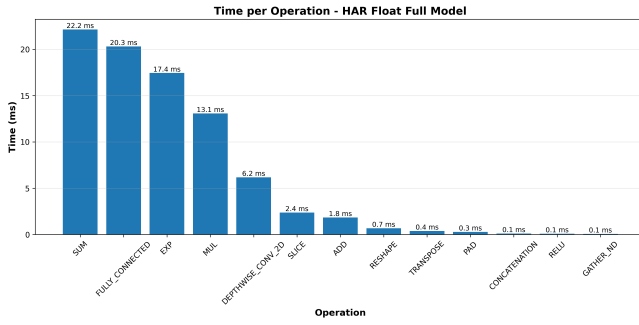
Memory usage behaves as expected: INT8 variants require significantly less RAM (Figure 6). The model split also reduces peak RAM usage. (Note: the split model currently uses several large buffers outside the TFLM allocator that are not yet included in the reported graph.)

For the split models we are also able to measure peak memory by inference stage (Figure 7). We see that by far the biggest bottleneck is the StepSSM section. (TODO: this changes to the first stage for KWS, and the non TFLM buffers are not counted)

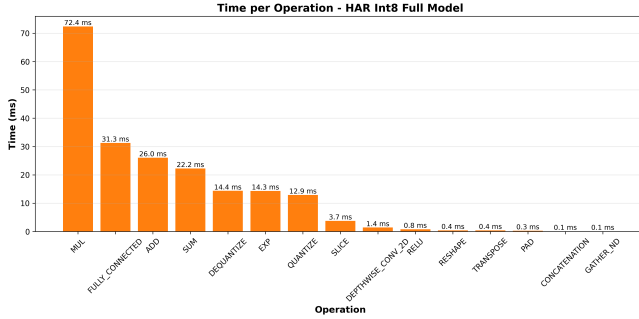
5.3 Discussion

The optimizations reduce memory at the cost of modest accuracy loss in some cases. Quantization is viable: for the full HAR model the accuracy drop is small (0.3%). The split model is more sensitive to quantization, possibly for reasons such as:

- static quantization achieving a better fit for the fully unrolled graph



(a) Float model



(b) INT8 model

Figure 5: Operation-type latency breakdown for the full models.

- numerical error accumulating more quickly in our SSM loop implementation

The unrolled graph can use different scales and zero points for different instances of the same operation, which may explain why the full (unrolled) model performs better under static quantization.

Our Mamba models—though smaller than state-of-the-art large models—achieve satisfactory accuracy: nearly 94% on HAR and 89% on KWS in the configurations tested, demonstrating Mamba’s potential for compact tasks.

We see however that the split of the model allows us to run more complex models, as we did not manage to run the KWS model without this operation. This was partially caused by the limitations of the TFLM, with the concatenate operation allowing only up to 10 vectors to be connected at once, while for KWS it would need to concatenate 51 time steps. A surprising outcome of the split was that it also reduces the peak memory usage of the model. This is most likely caused by the allocation style used by TFLM, which is based on a greedy algorithm, and we suspect the split was able to optimize this decision. If true, however, splitting might not always have the same effect.

We observe that much of the latency arises from SUM, MUL, ADD, and EXP operations in the SSM layer. From a latency perspective, the SSM does not map efficiently to TFLM as-is, in part because of costly exponentiation on MCUs. Implementing a custom TFLM SSM kernel (possibly quantized) would likely yield substantial speedups; time constraints prevented us from implementing this in the cur-

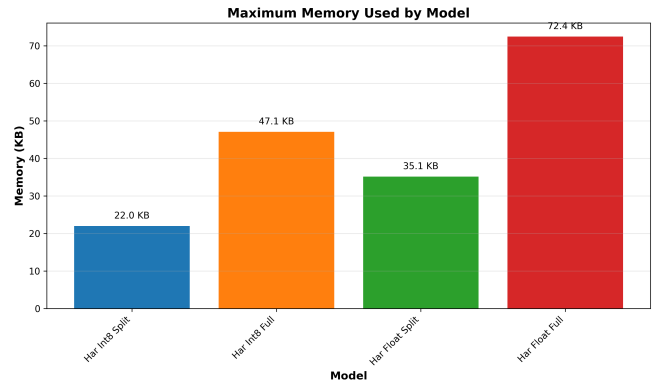


Figure 6: Graph of peak RAM usage of the HAR models

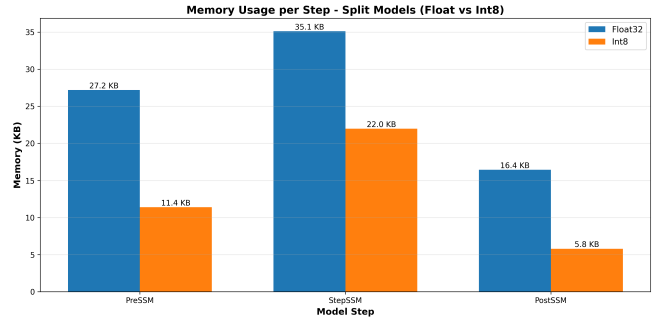


Figure 7: Graph of the peak RAM usage by inference stage

rent work.

5.4 Limitations

There are several limitations. Tooling for deploying Mamba on MCUs is immature, and time constraints limited our exploration.

We evaluated only two popular TinyML datasets, which do not cover use cases such as image/video tasks where Mamba may also be useful. We tested FLOAT32 and INT8 formats; exploring lower bit widths could be valuable but is currently limited by TFLM operator support.

Results were gathered on a single MCU family (ESP32); different vendors’ hardware acceleration choices may yield substantially different latency and performance characteristics.

Finally, our split approach complicates deployment because it requires model-aware inference code. Implementing the SSM as a custom, quantized TFLM kernel would preserve deployment simplicity while enabling the same memory/time trade-offs; we leave that for future work.

6 Responsible Research

This section discusses ethical considerations, reproducibility practices, and how AI tools were used during the research. It documents dataset and code licensing, replication steps, and potential misuse risks and mitigations.

6.1 Ethical Considerations

While this work does not involve third-party participants, it is important to consider potential impacts. The main focus is enabling more efficient machine learning on microcontrollers. Shifting some computation from datacenters to low-power devices can improve sustainability.

On-device processing can cut cloud exposure and improve privacy by keeping raw data local, but it can also be misused (for example, for covert surveillance). Deployment should therefore consider safeguards, transparency, and appropriate governance.

6.2 Reproducibility

The datasets used in this research are publicly available, and all software used had licenses permitting research use and has been acknowledged.

We documented the steps needed to replicate the experiments, including model training, conversion, quantization, and microcontroller benchmarking. Full replication requires the specific MCU, but the ESP32 family used here is widely available in the TinyML community.

We used a single profiler consistently across runs and annotated results with the MCU variant used to improve comparability.

The project code is publicly available and includes instructions and library versions to facilitate replication.

6.3 Use of AI Tools

AI tools were used as supportive instruments during development. Their use was limited to writing helper scripts, assisting with debugging, and improving clarity and grammar in written text.

Specifically, AI assistance was used for:

- Writing scripts to set up the experiment pipeline
- Debugging implementation issues in experimental code
- Improving clarity and structure of written explanations
- Assisting with LaTeX formatting and consistency

All scientific decisions, experimental design choices, and interpretation of results were made independently by the author. AI tools were not used to generate or alter experimental outcomes, results, or conclusions.

7 Conclusions and Future Work

This paper examined how Mamba can be adapted and optimized for inference on microcontrollers. Our experiments support three concise findings:

- Deploying via TFLM (with model-graph stripping) substantially reduces peak RAM compared to the C-based MambaLite-Micro workflow (roughly a 75% reduction in our runs).
- Memory vs. latency trade-off: INT8 quantization cuts peak RAM by about $\approx 33\%$ and model file size by $\approx 60\%$, but on the tested ESP32-class hardware it increased latency because efficient INT8 vector instructions are lacking.

- Split vs. full deployment: splitting into pre-SSM / SSM-step / post-SSM lowers peak RAM and permits longer sequences, but increases sensitivity to static quantization and complicates the runtime pipeline.

Taken together, Mamba is a strong candidate for TinyML on MCUs when memory optimizations are prioritized, but real-world latency depends heavily on hardware and kernel support. A next step that could be pursued is implementing a custom, quantized Selective State-Space Model kernel for TFLM and running targeted microbenchmarks across MCU families to guide low-level optimizations. We also recommend systematically evaluating pruning methods (structured and unstructured) to reduce parameter count and memory footprint, and combining those experiments with Quantization-Aware Training and lower-bit formats.

References

- [1] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces.”
- [2] Z. Xu, Y. Yue, X. Hu, Z. Yuan, Z. Jiang, Z. Chen, J. Yu, C. Xu, S. Zhou, and D. Yang, “MambaQuant: Quantizing the Mamba Family with Variance Aligned Rotation Methods.”
- [3] I. F. Shihab, S. Akter, and A. Sharma, “Efficient Unstructured Pruning of Mamba State-Space Models for Resource-Constrained Environments,” *arXiv preprint*, 2025.
- [4] H. Xu, J. Xia, W. Yang, Y. Sui, and S. Xia, “MambaLite-Micro: Memory-Optimized Mamba Inference on MCUs.”
- [5] M. Tri Lê, P. Wolinski, and J. Arbel, “Efficient Neural Networks for Tiny Machine Learning: A Comprehensive Review,” vol. 17, no. 4, pp. 1–41.
- [6] T. Cassimon, S. Vanneste, S. Bosmans, S. Mercelis, and P. Hellinckx, “Designing resource-constrained neural networks using neural architecture search targeting embedded devices,” vol. 12, p. 100234.
- [7] K. Dokic, M. Martinovic, and D. Mandusic, “Inference speed and quantisation of neural networks with TensorFlow Lite for Microcontrollers framework,” in *2020 5th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM)*, pp. 1–6.
- [8] T. Suwannaphong, F. Jovan, I. Craddock, and R. McConville, “Optimising TinyML with quantization and distillation of transformer and mamba models for indoor localisation on edge devices,” vol. 15, no. 1, p. 10081.
- [9] A. Tegen, N. Lehmann, Y. Li, A. Cossettini, L. Benini, and T. M. Ingolfsson, “FEMBA on the Edge: Physiologically-Aware Pre-Training, Quantization, and Deployment of a Bidirectional Mamba EEG Foundation Model on an Ultra-low Power Microcontroller.”
- [10] C. Li, D. Huang, K. Yao, X. Ni, L. Shen, and F. Luo, “Physics-Guided Tiny-Mamba Transformer for Reliability-Aware Early Fault Warning.”

- [11] X. Ma, Z. Ni, and X. Chen, “TinyViM: Frequency Decoupling for Tiny Hybrid Vision Mamba.”
- [12] T. Wang, Y. Shu, L. Qiao, D. Ding, and G. Li, “Light-Mamba: A Resource-Efficient Deep-Learning Method for UWB NLOS Identification Based on Selective State-Space Modeling,” vol. 13, no. 5, pp. 8735–8748.
- [13] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, “TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems.”
- [14] M. Nachin, D. Desai, S. S. Jia, C. Lai, M. Liu, J. Szwejbka, R. Alvarez, R. Ascani, D. Bort, M. Candales, *et al.*, “ExecuTorch - a unified PyTorch solution to run AI models on-device,” *arXiv preprint arXiv:2605.08195*, 2026.
- [15] G. Franco, A. Pappalardo, and N. J. Fraser, “Xilinx/brevitas,” 2025.
- [16] A. Or, A. Jain, D. Vega-Myhre, J. Cai, C. D. Hernandez, Z. Zheng, D. Guessous, V. Kuznetsov, C. Puhersch, M. Saroufim, S. Rao, T. Tran, and A. Samardžić, “TorchAO: PyTorch-Native Training-to-Serving Model Optimization.”
- [17] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, “A Public Domain Dataset for Human Activity Recognition Using Smartphones,”
- [18] P. Warden, “Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition,” *ArXiv e-prints*, Apr. 2018.
- [19] Y. Enokibori, “rTsfNet: A DNN model with Multi-head 3D Rotation and Time Series Feature Extraction for IMU-based Human Activity Recognition.”
- [20] J. Wang, L. Yu, L. Huang, C. Zhou, H. Zhang, Z. Song, Z. Ma, and Z. Zhang, “Efficient Speech Command Recognition Leveraging Spiking Neural Network and Curriculum Learning-based Knowledge Distillation,”